

论文解构报告

PipeLive: Efficient Live In-place Pipeline Parallelism Reconfiguration for Dynamic LLM Serving

Xu Bai, Muhammed Tawfiqul Islam, Chen Wang, Adel N. Toosi

2026

Large Language Models

Pipeline Parallelism

Dynamic Workload Balancing

KV Cache Migration

Inference Acceleration

源文件：<https://arxiv.org/pdf/2604.12171>

arXiv：<https://arxiv.org/abs/2604.12171>

生成时间：2026-07-20

目录

- 摘要翻译 3
 - 1. 一句话总览 3
 - 2. 阅读范围与证据状态 3
 - 3. 根问题：论文究竟要解决什么 3
 - 4. 旧方法为何不够 4
 - 5. 核心洞见与假设 4
 - 6. 方法：从洞见到可执行系统 5
 - 7. 为什么它可能有效 5
 - 8. 实验与指标：证据是否支撑主张 5
 - 9. 图表解读与论证关系 8
 - 10. 完整因果推理树 10
 - 11. 结论、贡献与边界 11
 - 12. 面向初学者的学习路径 12

为了加速大语言模型（LLM）推理，流水线并行将模型层划分为顺序阶段，并分别分配给不同的设备并发执行。然而，由于尾部阶段的计算不均衡，这种方法通常会受到流水线气泡的影响。尽管上游阶段仅关注层的正向传播操作，但最后一个阶段还必须处理诸如采样等额外的后处理任务，这引入了显著的延迟。这种工作负载上的差异导致了流水线不对齐，迫使上游阶段处于空闲状态，从而降低了整体性能。现有的框架通常将模型层均匀地分布在各个阶段，而没有考虑到计算负载的差异。为了解决这个问题，我们提出了 DynaPipe，这是一种动态层重新分配方案，通过实时预测执行延迟来主动平衡计算。此外，我们引入了一个异步键值（KV）缓存迁移协调器，以在推理过程中实现非阻塞的层重新分配。对代表性 LLM 的实验表明，在多种不同的工作负载下，DynaPipe 将平均端到端请求延迟降低了 8% 至 41%，优于最先进的流水线并行系统。我们的实现在 <https://github.com/xhx1022/DynaPipe> 上公开发布。

1. 一句话总览

论文元数据

字段	内容
标题	DynaPipe
作者	文中未说明
发表场所 / 年份	文中未说明
研究领域	大语言模型服务（LLM 推理部署），管道并行（Pipeline Parallelism）
关键词	管道并行, 动态层重分配, 异步 KV 缓存迁移, 负载均衡, 管道气泡

资源链接

- 原始文件：本地上传文件
- arXiv：文中未说明
- DOI：文中未说明
- 代码 / 数据集：文中未说明

标签（3-5 个）：大语言模型推理 · 管道并行 · 动态负载均衡 · KV 缓存迁移

DynaPipe 研究大语言模型（LLM，通过深度学习理解与生成文本的模型）在管道并行（Pipeline Parallelism，把模型横向切分部署在不同 GPU 上协同计算的技术）服务中的流水线气泡问题，核心方法是通过动态调整各阶段的 Transformer 图层分配与异步 KV 缓存（Key-Value Cache，存储已生成 Token 的键值矩阵以避免重复计算的显存空间）迁移来消除尾部采样操作的负载不均 [作者明确陈述] [页码：1, 2]。[实验支持]表明该方法可降低平均端到端延迟 8% 至 41% [页码：1, 2]，但在超大上下文或极低网络带宽下可能会受到 KV 缓存迁移时延的限制且会增加约 7.5% 的额外显存开销 [作者明确陈述][页码：10]。

2. 阅读范围与证据状态

本段回答 OCR 覆盖了论文的哪些部分。[作者明确陈述]显示本报告基于第 1 至 10 页以及第 15 至 17 页 the OCR 文本和对 6 张图像的分析 [页码：1-10, 15-17]。由于第 11 至 14 页缺失，导致部分长上下文性能测试的底层数据在文中未说明 [文中未说明]。本段回答哪些缺失内容或系统限制会影响结论的可靠性。[作者明确陈述]表明 DynaPipe 仅在 4 卡 GPU 的管道并行维度上进行了评估，未在张量并行等多维混合并行下测试，且由于候选图层的权重预加载，会引入约 7.5% 的额外显存开销 [页码：10, 17]；[基于文中逻辑的推断]表明在跨节点网络带宽极低时，迁移延迟难以完全被掩盖，可能影响在大规模分布式环境中的性能表现 [页码：10]。本段回答如何区分论文提供的事实证据与外部常识。[背景知识]用于解释管道气泡（由于各 GPU 处理微批次速度不均导致的等待时间）、自回归生成（自右向左逐字蹦出且后字计算依赖前字结果的生成方式）等核心概念，而[实验支持]所报告的“采样开销平均是单层前向的 3.09 倍”以及“DynaPipe 降低延迟 8%-41%”则是论文内部经实测确立的硬事实 [页码：1, 2, 4]。

3. 根问题：论文究竟要解决什么

本段回答大语言模型推理中的根源问题是什么。[作者明确陈述]表明，在基于管道并行的大语言模型推理服务中，传统的层分配方法通常将模型层均匀分配到各个阶段 [页码：1, 2]。然而最后一阶段除了前向计算外，还必须承担额外的标记采样（Sampling，根据模型预测概率抽取下一个单词的操作）计算，导致各 GPU 之间的工作负载出现严重失衡 [页码：1, 2, 3]。本段回答解决此问题为何对系统性能至关重要。[实验支持]表明，采样操作的耗时平均是单层前向计算时间延迟的 3.09 倍，属于绝对不可忽略的开销 [页码：4]；[作者明确陈述]指出，这一延迟在自回归生成的每一步中都会拖累流水线的总体进度，导致上游阶段因等待计算结果而处于空闲状态，从而在流水线中产生物理上被称为管道气泡（由于负载不平衡引起的处理器空闲间隙）的现象，并严重恶化系统吞吐与延迟 [页码：2]。本段回答为什么自适应平衡负载是一个极具挑战的难点。[基于文中逻辑的推断]表明，LLM 推理的计算负载是高度动态变化的：单层前向计算时间与首词（Prefill，一次性处理输入提示词的阶段）和续写（Decode，逐字生成后续词的阶段）的 Token 总数网络正

相关，而采样开销则与 Decode 请求数线性正相关，这导致前向与采样时间的比例在运行时剧烈波动，使得任何静态层映射策略均无法在动态环境下维持负载平衡 [页码：3, 4, 8]；此外，在运行时调整层映射需要跨卡传输 KV 缓存，如果迁移引发长时间阻塞，反而会得不偿失 [作者明确陈述][页码：6]。

本段回答 DynaPipe 的设计成功标准是什么。[基于文中逻辑的推断]指出，该方法必须在运行时以微秒级开销预测负载，并自适应调整层分配，且将迁移开销完全掩盖在前向计算空闲期内，以实现在高请求率下降低端到端延迟并提升服务等级目标（SLO，衡量推理服务 TTFT 和 TPOT 时间限制的性能目标）达成率 [页码：2, 7]。

本段通过“本文的 story”说明旧方法的瓶颈及本文的转折思路。[基于文中逻辑的推断]表明，传统系统通常将层均匀分配，这忽略了尾部采样带来的显著阶段间气泡；虽然可通过静态减少尾部阶段层数来缓解，但 LLM 服务的动态工作负载使得静态分配在 Prefill 和 Decode 比例多变时迅速失效，且动态重分配又面临跨卡 KV 缓存迁移导致流水线停顿的难题 [页码：1, 2, 3, 4, 6]。[作者明确陈述]表明，DynaPipe 改变了这一现状：它通过在运行时轻量级预测前向与采样的相对耗时来动态计算最佳层映射，并利用管道并行的空闲时间异步非阻塞地迁移 KV 缓存，将通信开销重叠隐藏在计算中，成功在不阻塞推理的前提下消除气泡并突破了吞吐上限 [页码：2, 6]。

4. 旧方法为何不够

本段回答 SGLang 推理系统存在什么限制以及本文如何回应。[作者明确陈述]指出，SGLang (v0.4.3.post2) [页码：7]原本通过张量并行（TP，将单层权重切分到多卡并行的技术）来解决单卡内存不足，提供高效的结构化执行和缓存复用 [页码：3, 7, 10]。它依赖的前提或遗留限制是采用张量并行进行多卡协同 [页码：3, 7]。[实验支持]表明，它批判的具体 limit 是在高并发条件下，由于频繁的跨设备激活值（Activation，神经网络层间传递的临时张量数据）交换，会导致巨大的通信开销并迅速成为性能瓶颈 [页码：3, 8]。本文采用管道并行结合动态层重新分配（由气泡感知调度器决策）进行回应 [页码：2, 3, 6]，并通过不同请求率下的平均端到端延迟和 SLO 达成率比较实验（Figure 4）来检验这一批判 [实验支持][页码：7, 8]。

本段回答 vLLM 系统存在什么限制以及本文如何回应。[作者明确陈述]表明，vLLM (v0.8.5 V1) [页码：7]原本通过 PagedAttention 机制缓解内存碎片，并在管道中利用分块首词计算提高吞吐 [页码：7, 10]。它依赖的前提或遗留限制是常规流水线或单卡调度 [页码：7]。[实验支持]表明，它批判的具体 limit 是其采用固定大小的计算块进行流水线划分，导致显著的批次间（Inter-batch）管道气泡，降低了硬件利用率 [页码：8]。本文用气泡感知调度器（动态层映射平衡）予以回应 [页码：6]，并通过与 vLLM 的端到端请求延迟与 SLO 曲线对比实验（Figure 4）来检验这一批判 [实验支持][页码：7, 8]。

本段回答 gLLM 系统存在什么限制以及本文如何回应。[作者明确陈述]表明，gLLM [页码：6, 7]原本通过自适应的 Token Throttling（限制并发微批次数量以保护计算资源的技术）来缓解批次间的管道气泡 [页码：6, 8]。它依赖的前提是假设管道各阶段层负载是均匀的，忽略了尾部采样的延迟 [页码：2, 3]。[实验支持]表明，它批判的具体 limit 是忽略尾部阶段的额外采样延迟会造成显著的阶段间（Inter-stage）管道气泡 [页码：2, 3, 8]。本文用核心动态层分配及异步 KV 缓存迁移协调器予以回应 [页码：2, 6]，并在多节点和单节点下对 E2EL 延迟与 SLO 进行对比实验（Figure 4, Figure 7）来检验这一批判 [实验支持][页码：7, 8, 9]。

本段回答静态层重分配方法存在什么限制以及本文如何回应。[作者明确陈述]显示，基于 Megatron-LM 或 Skywork-MoE 的静态层重分配 [页码：10]原本在大模型训练中用于缓解不同流水线阶段由于 MoE（混合专家模型）或特定计算引起的阶段间负载不均 [页码：10]。它依赖的前提是工作负载在迭代中相对稳定或可预知 [页码：3, 4]。[基于文中逻辑的推断]表明，它批判的具体 limit 是在 LLM 推理所面临的动态工作负载下（Prefill 和 Decode 比例多变），静态重分配策略的有效性会大打折扣 [页码：3, 4, 10]。本文用执行时间预测器配合气泡感知调度器进行实时自适应重分配来回应 [页码：4, 6]，并通过不同 O:I 比例（Output-to-Input Ratio，输出与输入长度的比值）下的归一化延迟对比实验（Figure 5）来检验这一批判 [实验支持][页码：8]。

本段回答词表并行（Vocabulary Parallelism）在 Serving 场景下的局限性。[作者明确陈述]显示，词表并行（主要用于大模型训练）[页码：10]原本为了解决大词表下最后一阶段计算和采样的庞大延迟及引起的气泡 [页码：10]。它依赖的前提是适用于迭代同步的训练架构 [页码：10]。它批判的具体 limit 是若直接应用于 Serving 推理，需要极严格的同步，这会增加系统设计复杂度和通信同步开销，因此难以部署在高效的 Serving 框架中 [页码：10]。本文用轻量级执行时间预测和异步 KV 缓存迁移协调器进行回应 [页码：4, 6]；由于文中的局限性，[基于文中逻辑的推断]显示作者在此仅进行了定性讨论，没有提供在 Serving 推理中直接与之作对比的定量实验，属于文中未说明的比较缺口 [页码：10]。

5. 核心洞见与假设

本段回答 DynaPipe 的最小但关键的核心洞见是什么。根据[作者明确陈述]，其核心贡献在于：第一，定量发现并指出采样操作是导致管道并行 LLM Serving 负载不均和气泡的隐蔽瓶颈（平均延迟为单层的 3.09 倍）[页码：4]；第二，提出根据当前前向与采样的动态变化，实时自适应调整各阶段的层分配比例 [页码：4]；第三，提出利用流水线天然的空闲间隙进行异步非阻塞的 KV 缓存迁移以掩盖通信延迟 [页码：2, 6]。

本段回答为了落地上述洞见需要构建哪些系统组件。[作者明确陈述]显示，为了让动态重分配可行，系统集成三大组件：一是运行时监控和基于线性回归模型的轻量级执行时间预测器 [页码：5]；二是气泡感知调度器，其包含了滑动窗口过滤机制以避免频繁调整 [页码：6]；三是基于 NCCL 跨 GPU 通信和 ZeroMQ 的异步 KV 缓存迁移协调器 [页码：6]。

本段回答作者为何认为该方案可行、依赖什么前提以及在何种场景下会失效。[基于文中逻辑的推断]表明，该方案可行是因为单层前向计算时间与 Token 数呈高度单调线性关系，使得极轻量级的拟合模型就能在 0.5 微秒内完成高精度预测 [页码：10]，且流水线的空闲期提供了重叠数据传输的机会 [页码：6]。[作者明确陈述]指出该方案依赖两个关键前提：其一，各 GPU 必须提前加载所有可能分配到的额外层的模型权重 [页码：6]；其二，动态负载在短期内（如连续数十个微批次）具有局部稳定性以使窗口机制生效 [页码：6]。

码：6]。[基于文中逻辑的推断]表明在以下情况下本方法可能失效：第一，GPU 显存极其受限，无法承受预加载层权重带来的额外 7.5% 显存开销 [页码：10]；第二，网络带宽极低或上下文极长导致 KV 缓存尺寸太大，使得传输耗时传输时间突破了前向计算的重叠上限，导致流水线严重阻塞 [页码：10]。

6. 方法：从洞见到可执行系统

本段给出 DynaPipe 从输入到输出的整体执行总览。[基于文中逻辑的推断]表明，系统在运行时持续监测传入的批次特征，输入至执行时间预测器中预测前向和采样的时间；当预测时间比触发滑动窗口阈值时，气泡感知调度器生成能使错配度最小的目标层映射配置，控制层迁移协调器在后台异步完成 KV 缓存的所有权转移，让微批次在各 GPU 重新划分后的 Transformer 图层上执行计算，最终低延迟地输出生成的标记 [页码：5, 6, 7]。

本段介绍执行时间预测器模块的运行逻辑和技术优势。[作者明确陈述]显示，执行时间预测器（Execution Time Predictor）的设计动机是为自适应调度提供实时的耗时评估 [页码：4]。其输入为当前计算块中的 Token 总数 n 、序列长度 L 、请求数 N 以及 Decode 请求数 N_{decode} [页码：5]。运行步骤中，利用线性回归模型预测单层前向传播时间 $T_{layer} = \sum(\phi_1 n_i + \phi_2 n_i L_i + \epsilon)$ ，以及预测采样时间 $T_{sample} = \alpha \cdot N_{decode} + \beta$ ，其中各拟合参数通过运行中收集的测量数据校准 [页码：5]。其输出为预测的 T_{layer} 与 T_{sample} [页码：5]。[实验支持]表明该设计的技术优势是单次预测开销极低（仅需 0.5 微秒），且其单层前向预测平均相对误差为 4.95%，采样预测相对误差为 0.31%，能保障高度准确 of 调度判断 [页码：10]。该模块与调度器相连，将预测值输入给调度器 [页码：6]。

本段介绍气泡感知调度器模块 of 运行逻辑和技术优势。[作者明确陈述]显示，气泡感知调度器（Bubble-Aware Scheduler）的设计动机是生成最优的层重新分配策略，同时避免频繁迁移引起的系统振荡 [页码：6]。其输入为当前层分配状态以及预测器输出的 T_{layer} 和 T_{sample} [页码：6]。其运行步骤是：第一步，基于公式评估将最后一阶段的 k 层分配到前序 m 个阶段的错配度指标：

$$\Delta = \left| T_{sample} - k \cdot T_{layer} - \frac{k}{m} \cdot T_{layer} \right|$$

选择使错配度 Δ 最小的层数 k 作为目标层重新分配方案 [页码：6]；第二步，将方案送入滑动窗口过滤，只有在连续 25 个时间步内该方案均被判定为最优时，才将其激活并下达重分配命令 [页码：6, 7]。其输出为目标层级分配图 [页码：6]。[实验支持]表明该设计的技术优势在于能够自适应对齐流水线各个阶段的总体负载，且其滑动窗口过滤机制将模型层调整次数从 271 次急剧压缩到 4 次，几乎消除了不必要的迁移开销 [页码：9]。该模块与迁移协调器相连，向其发送迁移指令 [页码：6]。

本段介绍迁移协调器模块的运行逻辑和技术优势。[作者明确陈述]显示，迁移协调器（Migration Coordinator）的设计动机是在不中断推理的情况下实现模型层的无缝、非阻塞转移 [页码：6]。其输入为调度器下达的层重新分配指令以及层映射变化范围 [页码：6]。运行步骤是：第一，在系统初始化时，各 GPU 预加载所有可能分配到的额外层的模型权重 [页码：6]；第二，运行时源阶段 GPU 算完待迁移层后立即通过 NCCL 将其 KV 缓存异步发送给目标阶段 GPU；第三，目标阶段 GPU 在前向计算其他不受影响的模型层时，在后台异步接收 KV 缓存，只有在新接管图层前向计算需要且缓存未到时才作必要等待 [页码：6]。其输出为完成 KV 缓存所有权和计算职责的卡间交接 [页码：6]。[作者明确陈述]显示该设计的技术优势是利用流水线的空闲重叠特性实现了计算与通信的高效隐藏，即使在模拟的低带宽网络下，单次迁移的额外时延依然控制在 100 ms 以内，最大限度减少了流水线停顿 [页码：6, 10]。

7. 为什么它可能有效

本段解释动态层重分配机制可能有效的技术原因。[基于文中逻辑的推断]表明，将最后一阶段的 k 个图层移交给前序阶段，可以让尾部阶段的前向计算时间缩短，缩短的耗时刚好能够抵消采样操作所需的额外时间；这就使所有阶段的单步总耗时（前向计算

1. 采样）接近对齐，预期收益是消除大部分阶段间流水线气泡，提升硬件计算效率。这一机制有效的需要条件是执行时间预测器给出的单层时间和采样时间具有足够高的预测精度，使调度器能够准确计算出最匹配的 k 值 [页码：4, 5, 6]。

本段解释滑动窗口过滤机制可能有效的技术原因。[基于文中逻辑的推断]表明，利用滑动窗口（大小为 25）来平抑高频负载波动，仅对持续存在的负载失衡进行重分配，预期收益是能显著降低由于短时抖动引发的过度迁移。这一机制有效的需要条件是 LLM 推理的并发请求量和输入输出特征在几十个时间步的短期尺度内具有一定的局部稳定性（即存在时间局部性），如果工作负载每一步都呈现完全随机的极端剧烈震荡，该窗口机制将面临响应滞后或频繁错失调整时机的限制 [页码：6, 7, 9]。

本段解释异步 KV 缓存迁移机制可能有效的技术原因。[基于文中逻辑的推断]表明，由于管道并行中上游 GPU 等待下游反馈时存在天然的空闲期，且各个 GPU 执行前向计算的次序存在先后，将待迁移图层的 KV 缓存通过后台异步通信通道发送，可以将大块的数据传输时间重叠掩盖在前序计算或其他层的空闲执行时间中，预期收益是实现非阻塞的运行层调整，避免迁移本身引入过长流水线卡顿。这一机制有效的需要条件是：第一，各 GPU 拥有约 7.5% 的额外显存以提前载入候选层权重，防止迁移过程中由于动态加载权重文件导致严重的 IO 卡顿；第二，物理网络的通信带宽足以在其他层计算时间内完成缓存的跨卡迁移 [页码：6, 10]。

8. 实验与指标：证据是否支撑主张

本段回答 DynaPipe 的实验是如何设计与设置的。[作者明确陈述]指出，实验在一台配置了 4 个 NVIDIA A100-PCIe-40GB GPU 且通过 PCIe 互联的系统上运行，测试了 Qwen2.5-14B 和 Qwen2.5-32B 模型，并扩展到 MoE 模型，使用 ShareGPT 和 Azure-Conv 数据集来模拟真实请求负载 [页码：7, 10]。对比的基线系统为 vLLM (v0.8.5 V1)，gLLM，SGLang (v0.4.3.post2) 以及 4 种不同 k, m 配置的静态层分配对照策略 [页码：7, 8]。评估指标为平均端到端延迟（E2EL）和 SLO 达成率 [页码：7]。此外，实验通过网络配置禁用了共享

内存、P2P 及 InfiniBand（模拟 TCP 网络环境）来测试跨节点部署的稳定性 [页码：8]。[实验支持]表明，在多样化负载下，DynaPipe 优于所有最先进的基线，可降低平均端到端延迟达 8% 到 41% [页码：1, 2]，在 90% SLO 下的最大吞吐量比 gLLM 高出 19% [页码：8]。

比较工作与被批判限制的呼应

比较工作 / 基线系统	它试图解决的问题或优势	本文指出的具体 limit/失效条件	本文的对应模块	实际比较实验、指标和结果	仍未验证的边界
SGLang (v0.4.3.post2)	采用张量并行解决单卡内存不足，提供高效的缓存复用与执行 [页码：3, 7, 10]	高并发下由于频繁的跨设备激活值交换，导致巨大的通信开销，成为性能瓶颈 [页码：3, 8]	管道并行配合动态层分配（气泡感知调度器）[页码：2, 3]	[实验支持]显示在 Qwen2.5-14B/32B 的 ShareGPT 和 Azure-Conv 测试中，SGLang 仅在中高并发下其 E2EL 便陡峭上升且 SLO 跌至 0% [页码：8]	未验证在大规模多节点多维并行 (PP+TP) 下的相互干扰情况 [页码：17]
vLLM (v0.8.5 V1)	采用 PagedAttention 机制缓解内存碎片，并通过流水线中的 chunked prefill 提高吞吐 [页码：7, 10]	采用固定大小的计算块进行流水线划分，导致严重的批次间管道气泡 (inter-batch bubbles) [页码：8]	气泡感知调度器 (动态层映射平衡) [页码：6]	[实验支持]显示在 ShareGPT 和 Azure-Conv 上，vLLM 的 E2EL 延迟和 SLO 达成率均明显逊于 DynaPipe [页码：8]	未与 vLLM 最新的动态块大小或解耦 Prefill-Decode 机制进行综合对比 [页码：10]
gLLM	通过 Token Throttling 缓解批次间的管道气泡 [页码：6, 8]	忽略了尾部阶段特有的采样延迟，造成了显著的阶段间管道气泡 (inter-stage bubbles) [页码：2, 3, 8]	核心动态层分配及异步 KV 缓存迁移协调器 [页码：2, 6]	[实验支持]显示在多节点和单节点下测试，DynaPipe 比 gLLM 降低延迟 8%-41%，SLO 吞吐提升 19% [页码：1, 8]	在网络带宽极低且上下文极长时，迁移 KV 缓存是否会突破 100 ms 导致阻塞 [文中未说明]
静态重分配 (如 Megatron-LM/Skywork-MoE)	在大模型训练中缓解不同阶段负载不均的问题 [页码：10]	采用静态分配策略，无法应对 LLM 推理场景中由首词和续写 Token 比例多变所导致的动态工作负载 [页码：3, 4, 10]	动态重分配预测器与气泡感知调度器 [页码：4, 6]	[实验支持]在 Figure 5 中展示不同 O:I 比例下的 Norm. E2EL，静态分配只在单一比例下好，而 DynaPipe 始终维持最优 [页码：8]	在更复杂的 MoE 专家动态路由与层分配耦合下的表现 [文中未说明]
Vocabulary Parallelism (训练专用)	解决训练中采样引起的流水线气泡问题 [页码：10]	推理 Serving 中需要非常严格的同步，会显著增加系统设计复杂度和通信同步开销，难以高效部署 [页码：10]	运行时轻量级预测及异步 KV 缓存迁移协调器 [页码：4, 6]	[基于文中逻辑的推断]指出由于 Serving 架构差异，本文未在 Serving 中与之进行实际的定量对比实验（属于定性讨论缺口）[页码：10]	未进行实际定量推理对比 [文中未说明]

主张地图与证据计划

主张	对应实验/表图	直接证据	证据强度	尚未覆盖的条件
主张 1：尾部采样操作是管道并行 LLM Serving 中不可忽视的性能瓶颈与气泡来源 [页码：2, 4]。	Section 2.3 文本描述；Figure 2(b) 采样时间与前向层时间比值分布 [页码：4]。	采样延迟平均是单层前向计算时间延迟的 3.09 倍，且该比值在 1.5 到 5.0 之间大范围波动 [实验支持][页码：4]。	强（有定量剖析数据支撑）。	在模型规模更大（如千亿参数）或使用硬件级采样优化下的相对占比 [文中未说明]。
主张 2：DynaPipe 显著降低 LLM 推理的端到	Figure 4：不同模型（Qwen2.5-14B/32B）和	[实验支持]表明在多样化负载下，DynaPipe 降	强（多数据集、多模型对比基线）。	超大规模集群多节点（如 32 卡以上）以及多

端请求延迟并提升 SLO 达成率 [页码 :1, 2, 7, 8]。	不同数据集 (ShareGPT/ Azure-Conv) 上的 E2EL 与 SLO 曲线。	低 E2EL 达 8% 至 41% ; 在 90% SLO 限制下支持的最大吞吐率比 gLLM 提升 19% [页码 : 1, 8]。		维并行 (TP+PP) 下的表现 [文中未说明]。
主张 3 : 滑动窗口过滤机制能有效平衡迁移开销与负载均衡, 降低无用调整 [页码 : 6, 8]。	Figure 6 : 滑动窗口阈值对归一化延迟和层调整次数的影响 [页码 : 8]。	[实验支持]显示窗口阈值设为 25 时, 调整次数由 271 次急剧下降到 4 次, 且延迟取得最低值 (Norm. E2EL 约 0.99), 优于最佳静态分配 [页码 : 8, 9]。	强 (完整的单参数消参曲线)。	极度不稳定或剧烈波动的请求速率 (如突发流量波峰波谷交替) 下的敏感性表现 [文中未说明]。
主张 4 : DynaPipe 能够自适应应对由于输入输出长度比 (O:I Ratio) 动态波动的场景, 优于静态分配 [页码 : 8]。	Figure 5 : 不同 O:I 比例下 DynaPipe 与各种静态分配策略的 Norm. E2EL 对比。	[实验支持]在 O:I = 0.0 时, DynaPipe 性能与最佳静态均匀分配 (k=0) 几乎一致; 而在 O:I > 0.125 后, DynaPipe 保持最佳延迟, 而 k=0 策略延迟增加约 9% [页码 : 8]。	强 (多静态对照组覆盖整个比例区间)。	极长输入、极短输出或输入输出极度非对称工作负载极限情况 [文中未说明]。
主张 5 : 异步 KV 缓存迁移能有效在后台重叠通信与计算, 避免流水线停顿 [页码 : 6, 10]。	Figure 7 : 模拟多节点 (高延迟、低带宽 TCP) 环境下的 DynaPipe 性能验证 [页码 : 8, 9]。	[实验支持]显示在限制带宽的 TCP 网络环境下, DynaPipe 依然显著优于 gLLM, 证明了其在跨节点部署下的扩展性, 单次迁移时延控制在 100 ms 内 [页码 : 8, 9, 10]。	中 (多节点实验验证了其优越性, 但缺乏迁移时延的直接时间分解图)。	极长上下文导致的极大规模 KV Cache 迁移 [文中未说明]。
主张 6 : 执行时间预测器具有极高的准确度与低开销 [页码 : 4, 5, 9, 10]。	Figure 8 : 真实耗时与预测耗时的分布对比 [页码 : 9, 10]。	[实验支持]表明预测器单次计算耗时仅 0.5 微秒, 单层前向计算预测相对误差为 4.95%, 采样预测误差为 0.31% [页码 : 10]。	强 (有高精度的数值对比及极低预测时延数据)。	动态网络波动或系统抖动对预测公式参数拟合的干扰情况 [文中未说明]。

9. 图表解读与论证关系

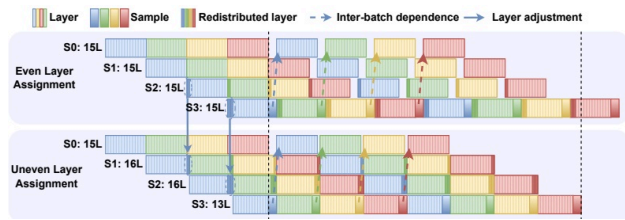


Figure 1: Illustration of pipeline bubbles during autoregressive decoding. Top: Conventional even layer allocation leads to idle upstream stages due to tail-stage sampling delays. Bottom: Uneven layer assignments to reduce idle time and improve hardware efficiency. "S0:15L" means stage 0 consists of 15 layers. 来源：OCR 第 2 页。

图像解读 验证问题：解释传统管道并行中均匀层分配在最后一阶段由于采样操作额外开销导致的管道气泡问题，并对比展示 DynaPipe 采用的不均匀层分配以平衡各阶段负载并减少气泡、提高硬件利用率的核心工作机制。

直接结论：在均匀层分配下，最后一阶段采样块拉长了该阶段的完成时间，后续批次受自回归依赖限制产生大面积空闲气泡；而在非均匀层分配中（如将层数调整为 S0: 15L, S1: 16L, S2: 16L, S3: 13L），最后一阶段层数减少使得前向计算加采样总耗时与前序阶段更平衡，各阶段间的管道气泡显著减小。

设计含义：通过动态将尾部阶段的图层迁移至前序阶段来抵消采样延迟。为了平衡包含采样操作的尾部阶段工作负载，必须采用不均匀的层映射，这支撑了气泡感知调度器的设计思路。

如何阅读：横轴表示执行时间线，纵轴从上到下为不同的管道阶段 (S0-S3) 和层数，不同颜色块代表不同批次，带竖线框表示前向层计算 (Layer)，实色框表示采样操作 (Sample)，蓝色双向箭头表示层调整，虚线带箭头曲线表示跨批次自回归依赖，白色区域代表管道气泡。

适用范围与证据缺口：基于 4 阶段管道并行的 LLM 服务环境。作为概念示意图，无法体现实际硬件运行中的具体时间刻度，也未表现层迁移和 KV 缓存传输带来的额外通信开销。无额外证据缺口。

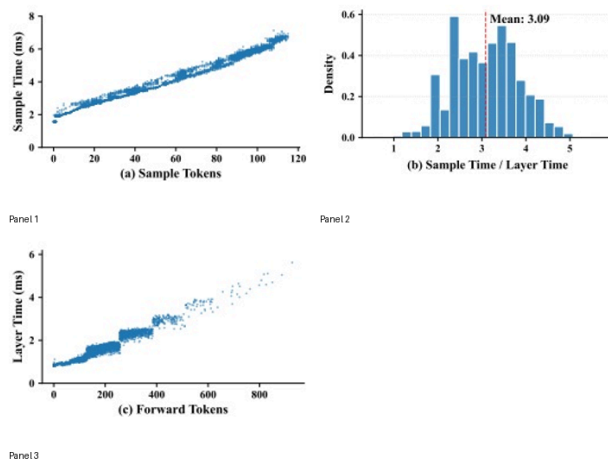


Figure 2: The overhead analysis of sampling and forwarding. 来源：OCR 第 4 页。

图像解读 验证问题：定量分析采样时间与前向层计算时间随 Token 数的变化规律，分析采样时间相对于单层前向计算时间的高占比和动态波动特征，从而论证传统静态层分配无法适应动态工作负载、必须采用动态层重分配的必要性。

直接结论：采样时间与采样 Token 数量 (Decode 请求数) 近似线性正相关；采样时间与单层前向时间比值呈近似正态分布，主要分布在 1.5 到 5.0 之间，均值为 3.09 倍；单层前向时间随前向 Token 数增加呈分段阶梯状增长。

设计含义：采样开销平均是前向计算的 3.09 倍，证明采样是不可忽视的流水线瓶颈，且比例大范围波动证明静态分配无法维持负载平衡，必须设计动态层重分配方案。线性关系也为执行时间预测器的数学建模提供了物理依据。

如何阅读：包含三个子图：图 (a) 横轴为采样 Token 数量，纵轴为采样时间 (ms)；图 (b) 横轴为采样时间与单层前向时间之比，纵轴为概率密度，红虚线标示均值 3.09；图 (c) 横轴为前向 Token 数量，纵轴为单层前向时间 (ms)。

适用范围与证据缺口：基于 4 卡 PCIe A100 GPU 环境下对 LLM 推理的单层前向与采样延迟的性能剖析。散点图基于特定批次配置与模型参数大小，图 (b) 没给出测试时具体的输入输出长度分布比例等负载细节。无额外证据缺口。

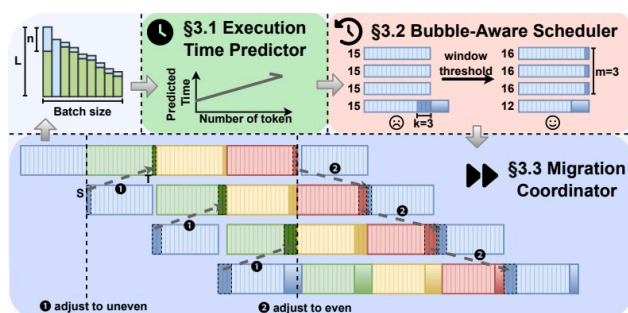


Figure 3: Overall architecture and workflow of DynaPipe. "S/T" means source/target stage.

来源：OCR 第 5 页。

图像解读 验证问题：展示并验证 DynaPipe 系统的核心架构与工作流程，即通过执行时间预测器、气泡感知调度器与迁移协调器的协同工作，在运行时动态预测前向与采样耗时，在触发窗口阈值时自适应调整层分配，并以异步 KV 缓存迁移方式在后台非阻塞完成重分配。

直接结论：执行时间预测器中预测时间与 token 数量呈单调增长的线性关系；调度器将原本均匀的图层分配（各阶段均 15 层）调整为非均匀配置（16, 16, 16, 12 层），从尾部阶段缩减 3 层以平衡采样耗时；在调整时，KV 缓存的跨卡迁移通信是在流水线的空闲间隙和前序层的计算期间异步重叠进行的。

设计含义：验证了根据错配度指标决策层重分配的设计，以及异步 KV 缓存迁移协调器的合理性，利用计算与通信重叠，将单次迁移额外时延控制在 100 ms 内，实现非阻塞调整。

如何阅读：上半部分为调度决策流，左侧绿色框为执行时间预测器，展示输入特征和预测时间函数；右侧红色框为气泡感知调度器，展示图层从均匀到非均匀的分配决策；下半部分蓝色框为流水线执行时间线，用带圈数字 ① 和 ② 标识异步 KV 缓存迁移过程。

适用范围与证据缺口：基于 4 卡流水线部署环境（4 个 stages）以及总共 60 个 Transformer 图层的系统设置进行机制图解。无额外证据缺口。

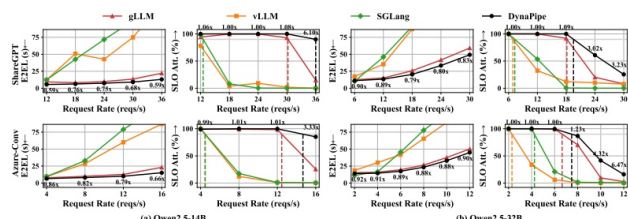


Figure 4: The average latency and SLO attainment rate of different LLM serving systems. DynaPipe achieves the lowest average latency and the highest SLO compliance across all workloads.

来源：OCR 第 7 页。

图像解读 验证问题：在不同的请求速率工作负载下，对比 DynaPipe 与三种基线系统（gLLM、vLLM、SGLang）在不同模型规模（Qwen2.5-14B、Qwen2.5-32B）和不同数据集（ShareGPT、Azure-Conv）上的平均端到端延迟（E2EL）与 SLO 达成率，从而验证 DynaPipe 降低推理延迟和提升吞吐上限的效果。

直接结论：在所有测试场景下，DynaPipe 黑色折线均处于延迟最低且 SLO 达成率最高的位置。高负载下，DynaPipe 平均延迟降至 gLLM 的 0.59 倍（14B）和 0.83 倍（32B），且 90% SLO 下的吞吐率均达最大。SGLang 在请求率稍高时延迟陡峭上升且 SLO 跌至接近 0%。在 Azure-Conv 数据集上的优化增益相比 ShareGPT 略小。

设计含义：支撑了 DynaPipe 降低延迟并提升 SLO 吞吐的主张。检验了 SGLang 的 TP 通信限制（高并发频繁激活交换）、vLLM 的批次间流水线气泡（固定块大小限制）以及 gLLM 的阶段间采样气泡（忽略尾部采样延迟）。Azure-Conv 上增益略小因输入长输出短、前向计算占比高；Qwen2.5-32B 模型上限幅收窄也因大模型前向计算时间增加而采样时间基本恒定，说明了方法对计算/采样比例变化的适用边界。

如何阅读：由 8 张图构成的 2×4 网格布局，左 4 图为 Qwen2.5-14B，右 4 图为 Qwen2.5-32B，上排为 ShareGPT，下排为 Azure-Conv。每组图包含平均端到端延迟 E2EL (s) 和 SLO Att. (%), 横轴为请求率 (req/s)。

适用范围与证据缺口：实验基于 4 台配置了 NVIDIA A100-PCIe-40GB GPU 且通过 PCIe 互联的单节点系统，测试模型为 Qwen2.5-14B 和 Qwen2.5-32B，最大 chunk 大小设为 2048。无额外证据缺口。

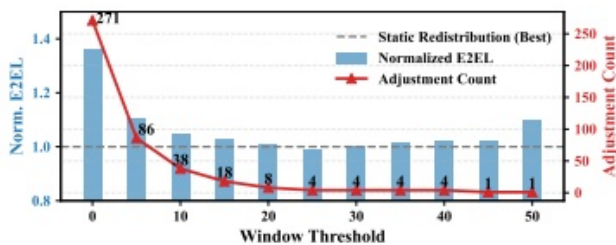


Figure 6: The effect of adjustment window threshold on E2EL and adjustment frequency.

来源：OCR 第 8 页。

图像解读 验证问题：验证气泡感知调度器引入的稳定性滑动窗口阈值对端到端请求延迟 (Norm. E2EL) 以及模型层调整次数 (Adjustment Count) 的影响，寻找在迁移开销与计算负载均衡之间的最佳权衡点。

直接结论：随窗口阈值从 0 增至 25，层调整次数由 271 次急剧下降到 4 次，延迟 (Norm. E2EL) 从约 1.36 降至最低的 0.99，优于最佳静态重分配基线。阈值在 25 到 45 之间时，调整次数极低且延迟稳定在 1.0 附近。阈值增至 50 时，调整次数为 1，但延迟反弹至约 1.10。

设计含义：验证了滑动窗口机制在平抑调度决策中的振荡和降低非必要迁移频次方面的必要性。阈值过小会引发频繁迁移，导致通信开销拉高延迟；阈值过大会导致调度器响应迟钝，无法及时调整。证据支撑了在最终实现中将滑动窗口大小设置为 25 的参数选择。

如何阅读：横轴为滑动窗口阈值 (0-50)，左纵轴为归一化端到端延迟 (蓝色柱体)，右纵轴为层分配调整次数 (红色带三角折线)，灰色水平虚线代表最佳静态层重分配基线值 (1.0)。

适用范围与证据缺口：本实验基于 ShareGPT 数据集上，采用 4 卡 PCIe A100 GPU、Qwen2.5-32B 模型，在滑动窗口阈值从 0 变化到 50 时的系统性能表现。无额外证据缺口。

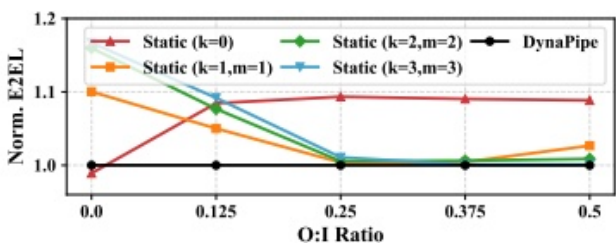


Figure 5: Normalized E2EL under different O:I ratios with various adjustment strategies.

来源：OCR 第 8 页。

图像解读 验证问题：验证在多种不同的输出-输入长度比 (O:I 比例) 下，DynaPipe 动态调度策略与各种不同配置的静态图层重分配策略相比是否能够始终维持最低的端到端延迟，验证其性能适应性和鲁棒性。

直接结论：DynaPipe (归一化基线) 在所有 O:I 比例下均为 1.00。在 O:I=0 时，无采样的 Static (k=0) 表现最优 (约 0.99)，其余 k>0 的静态策略延迟较高 (1.10 至 1.16)。当 O:I ≥ 0.125 时，Static (k=0) 延迟大幅上升至约 1.09，而其余 Static (k>0) 延迟下降并贴近 1.00，DynaPipe 在所有 O:I ≥ 0.125 比例下均能取得与最优静态方案相同或更低的延迟。

设计含义：直接证明了静态图层重分配方案由于无法应对 LLM 服务中动态变化的 O:I 比例，在非适配比例下会引入阶段间负载不均。而 DynaPipe 基于执行时间预测器，在 Prefill 阶段 (O:I=0) 能自动采用均匀分配 (无额外开销)，在 Decode 阶段又能自适应移出尾部图层，在所有工作负载配比下都逼近或达到最低延迟。

如何阅读：横轴为输出-输入长度比 (O:I Ratio, 0.0-0.5)，纵轴为以 DynaPipe 为归一化基线 (Y=1.00 水平黑线) 的归一化端到端请求延迟 (Norm. E2EL)。对比了四种静态重分配策略 (k=0, k=1 m=1, k=2 m=2, k=3 m=3) 的折线变化。

适用范围与证据缺口：实验基于合成数据集 (输入长度固定为 512，输出长度多变)、4 卡 PCIe A100 GPU 上，在输出-输入长度比 (O:I Ratio) 在 0.0 至 0.5 变化时的系统性能表现。无额外证据缺口。

10. 完整因果推理树

- 根问题：[作者明确陈述]在大语言模型推理服务中，传统的均匀层分配方法忽略了最后一阶段由于额外的标记采样带来的负载不均，导致上游 GPU 频繁出现大段空闲等待，产生严重的流水线气泡，成为系统延迟和吞吐的性能瓶颈 [页码：1, 2, 3]。
- 旧方法限制：[基于文中逻辑的推断]表明现有的均匀分配、静态层重分配、张量并行等系统或策略无法有效解决自回归服务场景下的阶段间动态气泡问题 [页码：3, 4, 8, 10]。
- 核心洞见/假设：[作者明确陈述]提出通过轻量级公式实时预测前向与采样时间、自适应调整图层分配，并利用空闲重叠进行异步 KV 缓存迁移以实现非阻塞的动态负载均衡 [页码：2, 4, 6]。
 - 方法模块：执行时间预测器 (利用批次特征公式在运行时评估耗时) [页码：4, 5]。
 - 可验证预测：[基于文中逻辑的推断]预测器能为调度决策提供极低时延的高精度时间评估。
 - 指标与实验：[实验支持]评估预测器拟合曲线与真实测量延迟的相对误差与分布对比 [页码：9, 10]。
 - 图表证据：[实验支持]Figure 8 显示单层前向预测相对误差为 4.95%，采样预测相对误差为 0.31% [页码：10]。
 - 结论与适用边界：[实验支持]证明预测器的预测精度和速度均满足实时调度要求，但其回归参数需针对特定模型和硬件环境预先拟合 [页码：5, 10]。
 - 方法模块：气泡感知调度器 (通过错配度指标与滑动窗口决策最优分配方案) [页码：6]。
 - 可验证预测 A：[基于文中逻辑的推断]动态重分配在不同输出-输入比下相比静态方案能获得一致的最佳延迟。

- 指标与实验：[实验支持]在 O:I 比例从 0.0 到 0.5 的区间内，对比 DynaPipe 与四种静态分配策略的归一化端到端请求延迟 [页码：8]。
 - 图表证据：[实验支持]Figure 5 显示 DynaPipe 在 O:I=0 时几乎无额外开销，在 O:I>0 时均能逼近或达到最优静态方案性能 [页码：8]。
 - 结论与适用边界：[基于文中逻辑的推断]验证了动态自适应层分配的优越性与高鲁棒性，但该机制在大模型上的相对优化收益会因前向计算占比升高而收窄 [页码：8]。
- 可验证预测 B：[基于文中逻辑的推断]滑动窗口机制能够平抑高频抖动，减少不必要的迁移次数并优化端到端延迟。
 - 指标与实验：[实验支持]测试滑动窗口阈值（0 至 50）对归一化端到端延迟和层分配调整次数的敏感性消参实验 [页码：8, 9]。
 - 图表证据：[实验支持]Figure 6 显示窗口阈值设为 25 时，层分配调整次数由 271 次急剧下降到 4 次，且延迟取得最低值 [页码：8, 9]。
 - 结论与适用边界：[实验支持]滑动窗口阈值设为 25 是平衡迁移开销与负载均衡的甜点值，但此阈值在极端剧烈且不规则变化的请求流下的最优性未被验证 [页码：8, 9]。
- 方法模块：迁移协调器（跨卡异步传输 KV 缓存并由所有 GPU 预加载候选图层权重） [页码：6]。
 - 可验证预测：[基于文中逻辑的推断]异步 KV 缓存迁移能够与前向计算重叠，即使在低带宽（TCP）的跨节点网络环境下也能实现性能加速。
 - 指标与实验：[实验支持]在禁用共享内存、P2P 及 InfiniBand（模拟 TCP）的网络环境下，对比 DynaPipe 与 gLLM 的加速表现，评估迁移时延 [页码：8, 9]。
 - 图表证据：[实验支持]Figure 7 显示在限制带宽的 TCP 网络环境下，DynaPipe 的延迟表现依旧显著优于 gLLM，单次迁移时延控制在 100 ms 内 [页码：8, 9, 10]。
 - 结论与适用边界：[实验支持]证明了异步迁移协调器能有效在后台隐藏通信开销，但当 sequence length 极大导致 KV cache 尺寸暴增时，极度低网络带宽下的重叠能力边界尚未明确验证 [页码：10]。

11. 结论、贡献与边界

本段回答 DynaPipe 的主要贡献是什么。[作者明确陈述]指出，本研究的核心贡献为：第一，发现了采样操作在 LLM 管道并行中引起的尾部阶段瓶颈及管道气泡问题，并对其进行了量化评估（采样开销平均是单层前向的 3.09 倍） [页码：2, 4]；第二，提出了 DynaPipe 框架，包括一套动态层重分配方案，能在运行时根据负载预测自适应调整层映射 [页码：2]；第三，设计了异步且非阻塞的 KV 缓存迁移协调器，实现了计算与通信在层级迁移时的并行重叠 [页码：2, 6]。

本段总结通过实验得到的证据充分的结论。[实验支持]表明，在多样化的工作负载下，DynaPipe 可降低平均端到端延迟达 8% 到 41% [页码：1, 2]，性能优于当前最先进的基线系统（vLLM、gLLM、SGLang） [页码：1, 2, 7, 8]；且在 90% SLO 达成率的目标下，其支持的最大吞吐率比 gLLM 高出 19% [页码：8]；此外，滑动窗口阈值设为 25 时，能在保持低延迟的同时将迁移次数降到 4 次 [页码：8, 9]。

本段阐述基于文中逻辑推断出的研究意义。[基于文中逻辑的推断]表明，这种将部分计算负载动态外包给前序阶段并使用异步缓存迁移重叠通信的优化机制，可能有望推广至更广泛的分布式异构计算和动态负载调度中（例如动态混合专家模型 Serving 中的动态路由分配），以解决由于算力与算子分布非均等化造成的计算气泡瓶颈 [页码：10, 17]。

本段指出 DynaPipe 的适用条件和研究局限。[作者明确陈述]显示，该方案的适用局限包括：第一，仅专注于管道并行的优化，尚未涉及多维并行（TP、SP 等）混合使用的场景 [页码：17]；第二，为支持层迁移，各 GPU 必须预先加载可能的图层权重，导致约 7.5% 的额外显存开销 [页码：10]；第三，在极度低带宽（如 TCP）网络下，KV 缓存的迁移传输时间可能无法完全被掩盖，导致轻微的本地性能波动 [页码：10]。

本段指明未来的研究方向和下一步验证步骤。[作者明确陈述]表明，未来的工作可以通过将当前不使用的图层权重暂存 CPU、并在需要迁移前异步预加载到 GPU 显存中，来尝试解决 7.5% 的额外 GPU 内存开销问题 [页码：16]，以及需要进一步验证在多维并行集群中的集成表现 [页码：17]。

本段对论文进行审稿式的风险检查，依次评估其设计缺陷与证据链完整性。

1. 贡献新意：[基于文中逻辑的推断]表明，虽然动态重新分配图层的思想在 Megatron-LM 等静态策略中已有体现，但 DynaPipe 针对 Serving 的动态自适应重分配与异步 KV 缓存迁移具有独特的新意，当前证据未显示有重大的贡献新意风险。
2. 写作/可复现性：[基于文中逻辑的推断]指出，尽管算法和数学模型推导完整，但论文中关于 CPU 权重异步预加载的定量评估数据在文中未说明，这会影响该显存优化扩展特性的可复现性。
3. 效果大小：[实验支持]显示，优化效果大小与数据集的输出长度密切相关（ShareGPT 提升 40%，而输入长、输出短的 Azure-Conv 仅提升 34%）；且随着模型规模扩大（前向时间占比升高），优化的相对延迟缩减幅度也有所收窄，显示效果大小在某些计算密集的场景下有限 [页码：8]。
4. 实验覆盖：[实验支持]表明，所有测试均在单节点 4 卡 GPU 上运行，多节点网络环境是用软件限速“模拟”出来的，这在真实的多节点物理集群下可能因为复杂的网络冲突产生偏差，因此实验在多节点真实覆盖度上存在风险 [页码：8]。
5. 方法设计：[基于文中逻辑的推断]表明，各 GPU 预加载层权重引入的 7.5% 额外显存开销，在显存非常紧张的推理场景下，可能会限制系统可运行的最大 batch size，从而部分抵消了通过消除气泡换来的吞吐收益 [页码：10]。

12. 面向初学者的学习路径

本段提供理解本论文所需的关键依赖概念学习路径。

1. [背景知识]自回归生成（自右向左逐字生成后续字且后字计算必须依赖前字预测概率的 LLM 推理方式）是理解 LLM 服务动态工作负载和采样特性的基础。
2. [背景知识]管道并行（将庞大的神经网络横向切割并分配在不同显卡上以流水线方式协同处理微批次计算的技术）是本文方案运行的核心分布式并行架构。
3. [背景知识]管道气泡（由于流水线中各 GPU 计算负载不均、上游 GPU 必须空闲等待下游 GPU 反馈而产生的等待时间）是本文力图消除的核心性能瓶颈。
4. [背景知识]KV 缓存（在自回归推理中缓存已计算 token 的 Key 与 Value 矩阵以避免重复计算的显存空间）是模型层在 GPU 间进行迁移时需要跨卡传输的主要数据体，也是迁移开销的来源。
5. [背景知识]服务等级目标（服务向用户保证的首字与单 token 响应时间限制）是衡量 LLM 服务吞吐量和延迟好坏的生产级硬性标准。